

■ POWER BI

ULTIMATE EXAM STUDY GUIDE

Data Visualization • DAX Functions • Power Query

Structured & Unstructured Data • Calendar Functions • Visuals

■ Every Syllabus Topic Covered | ■ Exam-Focused | ■ Zero Knowledge Assumed

Study hard. Score full marks. ■

■ Table of Contents

SECTION 1

- 1.1 Structured Data
- 1.2 Unstructured Data
- 1.3 Semi-Structured Data

SECTION 2

- 2.1 DAX IF / Nested IF
- 2.2 DAX SWITCH
- 2.3 Why SWITCH over Nested IF?

SECTION 3

- 3.1 Age Calculation
- 3.2 TAT (Turnaround Time) Calculation
- 3.3 Date Difference (DATEDIFF)
- 3.4 Date Conversions & FORMAT

SECTION 4

- 4.1 CALENDAR Function
- 4.2 CALENDARAUTO Function
- 4.3 Building a Date Table with Year/Quarter/Month

SECTION 5

- 5.1 Power Query: Import Mode
- 5.2 Power Query: Direct Query
- 5.3 Power Query: Native Query (SQL)
- 5.4 Power Query: Live Connection
- 5.5 Comparison Table

SECTION 6

- 6.1 Making Visuals in Power BI
- 6.2 Canvas Views (Report / Data / Model)
- 6.3 Making Relations & Relation View
- 6.4 Slicers (Date, Dropdown, Year)
- 6.5 Input Boxes & Limitations

SECTION 7

- 7.1 Age Groups (Nested IF & SWITCH)
- 7.2 Sales Amount Between Dates (CALCULATE)
- 7.3 Ugly, Bad & Wrong Visuals

SECTION 8 — PAST PAPER ANSWERS

- Task 1–10 Full Answers

SECTION 1 — Data Types: Structured, Unstructured & Semi-Structured

■ 1.1 Structured Data

■ The 2nd-Grader Explanation

Imagine a LEGO set where every piece has its own labelled box. You know exactly where each brick goes. Structured data is like that — everything is in rows and columns, neat and tidy, just like a school attendance sheet or an Excel spreadsheet.

■ The 'Full Marks' Details

- Structured data is organized into rows and columns (like a table).
- It is stored in Relational Databases (like Excel, SQL, MySQL, Oracle).
- Every column has a fixed data type (e.g., Name = Text, Age = Number).
- Examples: Microsoft Excel, Google Sheets, SQL Server, MySQL, PostgreSQL, Oracle, IBM DB2.
- BEST for Power BI — Power BI loves structured data!
- Easy to search, filter, and analyse.

■ Quick Quiz

Q: What type of data is stored in rows and columns like an Excel sheet?

A: Structured Data.

■ 1.2 Unstructured Data

■ The 2nd-Grader Explanation

Imagine you tip a box of random toys onto the floor — rubber ducks, marbles, toy cars, all mixed up. No labels, no order, nothing matches. That is unstructured data! Photos, videos, and audio files are unstructured because they are all different and cannot be stored in neat rows.

■ The 'Full Marks' Details

- Unstructured data has NO fixed format, no rows, no columns.
- It is ambiguous — no rhyme or reason.
- Examples: Photos, Videos, Audio files, Text files (like emails or Word docs).
- The data is NOT replicable — every photo/video is unique.
- Hard to directly analyse with Power BI without processing first.
- It may have METADATA (e.g., date a photo was taken) but the content itself is unstructured.

■ Quick Quiz

Q: Is a video file considered structured or unstructured data?

A: Unstructured — because its content (the video itself) is unique and cannot be put in rows/columns.

■ 1.3 Semi-Structured Data

■ The 2nd-Grader Explanation

Imagine a backpack where things are in bags-within-bags. It is not as neat as the LEGO set, but you can still find things using little tags. Semi-structured data is like that — it has TAGS to organise itself, but it does not follow a strict table format.

■ The 'Full Marks' Details

- Semi-structured data has SOME organisation using TAGS or LABELS.
- It is NOT stored in relational (row/column) systems.
- Best associated with NoSQL databases (non-relational).
- Examples: JSON files, XML files, HTML files.
- Serialized programming practices are used to read/write this data.
- It CAN be converted/parsed into structured data for use in Power BI.
- No hard-and-fast format — flexible structure.

KEY DIFFERENCE: Structured = table, Semi-structured = tagged but no strict table, Unstructured = no format at all.

■ Quick Quiz

Q: What is the main difference between semi-structured and unstructured data?

A: Semi-structured has TAGS for some organisation; unstructured has NO organisation at all.

SECTION 2 — DAX Functions: IF, Nested IF & SWITCH

■ 2.1 DAX IF Function

■ The 2nd-Grader Explanation

Think of IF like a traffic light. If the light is GREEN → go. If not → stop. DAX IF checks ONE condition and decides what to do.

■ The 'Full Marks' Details

- Syntax: IF(condition, value_if_true, value_if_false)
- Example: IF([Age] >= 18, "Adult", "Minor")
→ If Age is 18 or more → shows 'Adult', else shows 'Minor'.
- Returns ONE of TWO values based on a TRUE/FALSE test.
- Can be written as a Calculated Column or Measure in Power BI.

■ Quick Quiz

Q: Write a DAX IF that shows 'Pass' if Score >= 50, else 'Fail'.

A: IF([Score] >= 50, "Pass", "Fail")

■ 2.2 Nested IF (IF inside IF)

■ The 2nd-Grader Explanation

Nested IF is like multiple traffic lights on the same road. After passing the first light, you hit another one! Each IF adds another condition layer. It can get very long and confusing.

■ The 'Full Marks' Details

- Syntax: IF(cond1, val1, IF(cond2, val2, IF(cond3, val3, val4)))
- Example (Age Groups):
IF([Age]<13, "Child",
IF([Age]<18, "Teen",
IF([Age]<60, "Adult", "Senior")))
- Reads from OUTSIDE to INSIDE — first condition checked first.
- Works just like IF-ELSE IF chains in programming.
- DOWNSIDE: Gets very hard to read and maintain with many conditions.

■ Quick Quiz

Q: What happens when you put an IF inside another IF in DAX?

A: It creates Nested IF — the outer IF checks first; if false, the inner IF checks next.

■ 2.3 DAX SWITCH Function — Why use it?

■ The 2nd-Grader Explanation

SWITCH is like a remote control with many buttons. Each button (case) does something different. It is much cleaner than pressing button-1, and if not button-1 then button-2, and if not then button-3 — which is what Nested IF does.

■ The 'Full Marks' Details

- SWITCH replaces long, messy Nested IF chains — same result, cleaner code.

- Syntax Option 1 (value matching):

SWITCH(expression, val1, result1, val2, result2, ..., default)

- Syntax Option 2 (TRUE mode — most powerful):

SWITCH(TRUE(), condition1, result1, condition2, result2, ..., default)

- Example (Age Groups with SWITCH TRUE):

SWITCH(TRUE(),

[Age] < 13, "Child",

[Age] < 18, "Teen",

[Age] < 60, "Adult",

"Senior")

- WHY SWITCH over Nested IF?

1. Easier to READ — all cases visible at once.
2. Easier to MAINTAIN — add/remove cases without rewriting whole logic.
3. Executes top-to-bottom — first matching condition wins.
4. Less chance of bracket/parenthesis errors.

- NOTE: Every programming language solves the same problem in multiple ways — SWITCH, IF-ELSE, CASE all do the same job but differ in readability and use case.

■ Quick Quiz

Q: In DAX, which function is cleaner for checking 5+ conditions — Nested IF or SWITCH?

A: SWITCH — it is easier to read, write, and maintain than deeply nested IFs.

SECTION 3 — DAX Date Functions: Age, TAT, DATEDIFF & FORMAT

■ 3.1 Age Calculation in DAX

■ The 2nd-Grader Explanation

Your age is just TODAY minus your Birthday year. If today is 2025 and you were born in 2005, you are 20! DAX does the same maths automatically.

■ The 'Full Marks' Details

- Formula (Calculated Column):

Age = DATEDIFF([DateOfBirth], TODAY(), YEAR)

- OR using INT division approach:

Age = INT((TODAY() - [DateOfBirth]) / 365.25)

- TODAY() gives the current date automatically.
- YEAR interval returns full completed years.
- Use in a table to create an 'Age' column from a 'DOB' column.
- Handle MISSING/BLANK DOBs: IF(ISBLANK([DOB]), BLANK(), DATEDIFF([DOB],TODAY(),YEAR))

■ Quick Quiz

Q: What DAX function calculates the number of years between two dates?

A: DATEDIFF(StartDate, EndDate, YEAR)

■ 3.2 TAT (Turnaround Time) Calculation

■ The 2nd-Grader Explanation

TAT is like checking how long a pizza delivery takes. If you ordered at 6 PM and received it at 7 PM — the TAT is 1 hour. In hospitals or labs, TAT = how many days from receiving a sample to getting results.

■ The 'Full Marks' Details

- TAT = Date Validated / Resulted MINUS Date Received.

- DAX Formula:

TAT = DATEDIFF([ReceivedDate], [ValidatedDate], DAY)

- From the SQL in your notes:

SUM(ISNULL(DATEDIFF(day, StatusInProgressToProcessing, StatusValidateToValidated), 0))

→ This sums TAT for all records; ISNULL replaces NULL with 0.

- In Power BI DAX:

TAT_Days = DATEDIFF([StatusInProgressToProcessing], [StatusProcessingToValidated], DAY)

- Can be shown as average: Average TAT = AVERAGE(Table[TAT_Days])

■ Quick Quiz

Q: What does TAT stand for and how is it calculated?

A: Turnaround Time = EndDate minus StartDate (usually in days using DATEDIFF).

■ 3.3 DATEDIFF Function

■ The 2nd-Grader Explanation

DATEDIFF is like a ruler between two dates. You tell it the start date, the end date, and what unit to measure in (days, months, years) — and it tells you the exact number.

■ The 'Full Marks' Details

- Syntax: DATEDIFF(StartDate, EndDate, Interval)
- Intervals: DAY, MONTH, QUARTER, YEAR, WEEK, HOUR, MINUTE, SECOND
- Example: DATEDIFF(DATE(2020,1,1), TODAY(), YEAR) → Number of years since Jan 2020
- If EndDate is BEFORE StartDate → result is NEGATIVE.
- Returns an INTEGER (whole number).
- Common uses: Age, TAT, Days overdue, Days until deadline.

■ Quick Quiz

Q: What interval would you use in DATEDIFF to find how many months passed between two dates?

A: MONTH — e.g., DATEDIFF([Start], [End], MONTH)

■ 3.4 Date Conversions & FORMAT Function

■ The 2nd-Grader Explanation

Imagine you have a date like 2025-03-15 and your boss wants it to say 'Mar 2025'. The FORMAT function is like a date costume — it changes how the date LOOKS without changing what it IS.

■ The 'Full Marks' Details

- FORMAT Syntax: FORMAT(DateValue, "FormatString")
- Common Format Strings:
FORMAT([Date], "MMM YYYY") → Mar 2025
FORMAT([Date], "DD/MM/YYYY") → 15/03/2025

`FORMAT([Date], "MMMM")` → March

`FORMAT([Date], "YYYY")` → 2025

`FORMAT([Date], "Q")` → 1 (Quarter number)

- Year Column: Year = `YEAR( [Date] )`
- Month Column: Month = `MONTH( [Date] )` (returns 1–12)
- Month Name: MonthName = `FORMAT( [Date], "MMM" )`
- Quarter: Quarter = `"Q" & QUARTER( [Date] )`
- Month-Year: MonthYear = `FORMAT( [Date], "MMM YYYY" )`
- Changing column Data Type to Date in Power Query: Right-click column → Change Type → Date.

■ Quick Quiz

Q: Which DAX function changes how a date looks (e.g., shows 'Mar 2025')?

A: `FORMAT( [Date], "MMM YYYY" )`

SECTION 4 — CALENDAR & CALENDARAUTO Functions

■ 4.1 CALENDAR Function

■ The 2nd-Grader Explanation

CALENDAR is like printing a calendar for a range YOU choose. You say: 'Give me every single day from January 1, 2020 to December 31, 2025' — and it makes a table with each date as one row.

■ The 'Full Marks' Details

- Syntax: `CALENDAR( StartDate, EndDate )`
- Creates a table with a single column called `[Date]`.
- One row per day in the range.
- Example:

`DateTable = CALENDAR( DATE(2021,1,1), DATE(2025,12,31) )`

- You then ADD columns like Year, Month, Quarter to this table.
- Used when you KNOW the exact date range you need.
- Steps in Power BI:
 1. Home → New Table
 2. Type: `DateTable = CALENDAR( DATE(2021,1,1), TODAY() )`
 3. Add columns: `Year = YEAR([Date])`, `Month = FORMAT([Date], "MMM")`, etc.

■ Quick Quiz

Q: What does the CALENDAR function return?

A: A table with one column [Date], containing every date between the StartDate and EndDate.

■ 4.2 CALENDARAUTO Function

■ The 2nd-Grader Explanation

CALENDARAUTO is like a smart calendar that reads YOUR data and automatically picks the right start and end dates. You do not have to tell it the range — it figures it out by scanning all your date columns!

■ The 'Full Marks' Details

- Syntax: `CALENDARAUTO( [FiscalYearEndMonth] )`
- Automatically scans ALL date columns in your data model.
- Picks the EARLIEST date as start and LATEST date as end.
- Generates a full calendar spanning from Jan 1 of the minimum year to Dec 31 of the maximum year.

- Optional: FiscalYearEndMonth — e.g., CALENDARAUTO(6) means fiscal year ends in June.
 - Example: DataTable = CALENDARAUTO()
 - WHEN TO USE which?
 - CALENDAR: When you want a fixed specific range.
 - CALENDARAUTO: When you want the range to adjust automatically with your data.
- BOTH create a [Date] column. CALENDARAUTO is faster to write.

■ Quick Quiz

Q: What is the key difference between CALENDAR and CALENDARAUTO?

A: CALENDAR needs you to set the date range manually; CALENDARAUTO scans your data and sets the range automatically.

■ 4.3 Building a Full Date Table (Exam Task Steps)

■ The 2nd-Grader Explanation

Here is exactly what the exam task asks: make a Date Table with Year, Quarter, Month-Year, then make slicers. Follow these steps in Power BI and you will get full marks.

■ The 'Full Marks' Details

STEP-BY-STEP:

1. Create Excel file with columns: order_date, shipment_date, received_date, sales_amount.
2. Load it into Power BI (Get Data → Excel).
3. Create DataTable: Home → New Table →
 DataTable = CALENDAR(MIN(Sales[order_date]), MAX(Sales[order_date]))
4. Add Year column: Year = YEAR(DataTable[Date])
5. Add Quarter column: Quarter = "Q" & QUARTER(DataTable[Date])
6. Add MonthYear column: MonthYear = FORMAT(DataTable[Date], "MMM YYYY")
7. Create relationship: DataTable[Date] ↔ Sales[order_date]
8. Add Slicer visual → drag DataTable[Date] → change to Range (slider).
9. Add dropdown slicer for Year → drag DataTable[Year] → set Style = Dropdown.
10. Add dropdown slicer for Month → drag DataTable[MonthYear] → Dropdown.

■ Quick Quiz

Q: Which DAX formula creates a MonthYear column in the format 'Mar 2025'?

A: MonthYear = FORMAT(DataTable[Date], "MMM YYYY")

SECTION 5 — Power Query Data Import Methods

■ 5.1 Import Mode

■ The 2nd-Grader Explanation

Import is like **DOWNLOADING** a song to your phone. The music is **NOW** on your phone. Even if the internet goes off, you can still play it. Power BI copies all the data into your file.

■ The 'Full Marks' Details

- Data is **COPIED** into Power BI's memory (compressed storage called VertiPaq).
- Works **OFFLINE** — data does not need a live database connection.
- Fastest query performance — everything is in memory.
- Data needs **MANUAL REFRESH** to update (scheduled or manual).
- **USE WHEN:** Data is not huge, performance matters, offline access needed.
- **DISADVANTAGE:** File size grows; not real-time.

■ Quick Quiz

Q: In Import mode, where is the data stored?

A: Inside Power BI's memory (VertiPaq engine) — a compressed copy.

■ 5.2 Direct Query

■ The 2nd-Grader Explanation

DirectQuery is like **STREAMING** a song. It is not saved on your phone — every time you press play, it fetches it live from the internet. Power BI sends queries to the database every time you interact with a visual.

■ The 'Full Marks' Details

- Data is **NOT** imported — Power BI queries the database **LIVE** each time.
- Always shows **REAL-TIME** / up-to-date data.
- No data stored in Power BI file → smaller file size.
- Slower — every click on a visual = new query to the database.
- Requires **CONSTANT** connection to the database.
- Some DAX functions **NOT** supported in DirectQuery.
- **USE WHEN:** Data changes constantly, very large datasets, real-time needed.

■ Quick Quiz

Q: What is the main advantage of DirectQuery over Import mode?

A: DirectQuery shows real-time data; data is always up-to-date without manual refresh.

■ 5.3 Native Query (SQL)

■ The 2nd-Grader Explanation

Native Query is like giving your waiter (Power Query) a HANDWRITTEN special order instead of pointing at the menu. You write your own SQL code, and it runs EXACTLY that query against the database.

■ The 'Full Marks' Details

- A Native Query = YOUR OWN SQL written directly in Power BI.
- HOW TO DO IT in Power BI (Steps):
 1. Data → Get Data → From Databases → From SQL Server Database.
 2. Enter Server name and Database name.
 3. Click Advanced Options.
 4. Paste your SQL in the SQL Statement box.
 5. Click OK → approve the native query prompt → data loads.
- ADVANTAGE: You can write complex SQL (JOINS, CASE WHEN, etc.) before data enters Power BI.
- DISADVANTAGE: Security risk — if someone else wrote the query, it runs with YOUR credentials.
- Warning shown: 'Do you approve running this native query?' → must approve.
- Works in Import mode OR DirectQuery mode.

■ Quick Quiz

Q: What security warning appears when you run a Native Query written by someone else?

A: Power BI warns you that the query will run with YOUR credentials, and asks for approval.

■ 5.4 Live Connection

■ The 2nd-Grader Explanation

Live Connection is like watching a TV channel LIVE. You are not downloading anything — you are watching exactly what is happening RIGHT NOW on the broadcasting station (Analysis Services or Power BI Service).

■ The 'Full Marks' Details

- Live Connection connects to Analysis Services (SSAS) or Power BI Service datasets.
- You CANNOT change the data model in Power BI Desktop — it is read-only.
- All calculations and data stay on the server — Power BI is just the display.
- Best for enterprise scenarios with a central shared data model.

- Difference from DirectQuery: DirectQuery goes to raw DB; Live Connection goes to a pre-built model.
- USE WHEN: You need to share one single source of truth across many reports.

■ Quick Quiz

Q: What is one major limitation of Live Connection in Power BI Desktop?

A: You CANNOT modify the data model — it is read-only, controlled by the server.

5.5 Comparison Table — All 4 Import Methods

Feature	Import	Direct Query	Native Query	Live Connection
Data Stored?	Yes (copy)	No	Depends	No
Real-time?	No (refresh)	Yes	Depends	Yes
Speed	Fastest	Slower	Varies	Fast
File Size	Large	Small	Depends	Small
Internet Needed?	No (after load)	Yes	Yes	Yes
Modify Model?	Yes	Partial	Yes	No
SQL Control?	Some	Some	Full SQL	No

SECTION 6 — Making Visuals, Canvas Views & Relations

■ 6.1 Making Visuals in Power BI

■ The 2nd-Grader Explanation

Visuals are like DRAWINGS of your data. Instead of staring at boring numbers, Power BI draws bar charts, pie charts, and line graphs to make the numbers easy to understand at a glance.

■ The 'Full Marks' Details

- Click on any Visual icon in the Visualizations panel (right side).
- Drag a field from the Fields panel into the visual's Value/Axis/Legend boxes.
- Common Visuals: Bar Chart, Line Chart, Pie Chart, Card, Table, Matrix, Slicer.
- Always give your visual a TITLE (Format → Title).
- Good Visual Principles (from your notes):
 - Accurately represents data (no misleading scales).
 - Aesthetically pleasing (no jarring colours).
 - Clear and easy to understand.
 - Does NOT distort numbers (e.g., bars must start at 0).

■ Quick Quiz

Q: What makes a data visualization 'wrong' (not just ugly)?

A: A 'wrong' visual has mathematical errors — it misrepresents actual values (e.g., bars without a proper y-axis scale).

■ 6.2 Canvas Views: Report, Data & Model Views

■ The 2nd-Grader Explanation

Power BI has 3 screens like 3 rooms in a house: the living room (Report View) where you show charts, the storage room (Data View) where you see raw data tables, and the map room (Model View) where you connect your tables.

■ The 'Full Marks' Details

- REPORT VIEW (paintbrush icon): Where you build your visuals and dashboards. This is what the end user sees.
- DATA VIEW (table icon): Shows your raw data in a spreadsheet-like view. You can see rows and columns. Good for checking data and writing DAX columns.
- MODEL VIEW (diagram icon): Shows all your tables as boxes and the connections (relationships) between them. Like an ERD diagram.

- Switch between views using the icons on the LEFT sidebar.

■ Quick Quiz

Q: In which Power BI view do you BUILD charts and dashboards?

A: Report View (the paintbrush/canvas icon).

■ 6.3 Making Relations & Relation View

■ The 2nd-Grader Explanation

A Relation is like a bridge between two tables. Imagine one table has Customer IDs and another has Sales — the Customer ID column is the bridge connecting them. Without this bridge, Power BI cannot match customers to their sales.

■ The 'Full Marks' Details

- Go to MODEL VIEW (left sidebar).
- Drag one column from Table A onto the matching column in Table B → a line (relationship) is created.
- Relationship Types:
 - One-to-Many (1:*) : Most common. One customer → Many orders.
 - Many-to-One (*:1) : Same as above, from the other side.
 - One-to-One (1:1) : Rare. Each row matches exactly one row.
 - Many-to-Many (*:*) : Advanced. Avoid if possible.
- The connecting column MUST have matching values in both tables.
- Cross-filter Direction: Single or Both — controls how filters flow between tables.
- Use Model View → Manage Relationships to edit or delete relationships.

■ Quick Quiz

Q: How do you create a relationship between two tables in Power BI?

A: In Model View, drag the common column from one table and drop it onto the matching column in the other table.

■ 6.4 Slicers — Date Slicer, Dropdown Month & Year

■ The 2nd-Grader Explanation

A Slicer is like a FILTER button for your report. When you move it, all your charts update to show only the data you chose. It is the most interactive element for end users.

■ The 'Full Marks' Details

- Add Slicer: Click Slicer icon in Visualizations panel → drag a field into it.

- DATE SLICER (Range slider): Drag a Date column → click Format → Slicer Settings → Style = Between. Shows a sliding date range.
- DROPDOWN MONTH LIST: Drag MonthYear column (e.g., 'Mar 2025') → Format → Slicer Settings → Style = Dropdown.
- DROPDOWN YEAR LIST: Drag Year column → Format → Slicer Settings → Style = Dropdown.
- Task 8 (start/end date in separate visuals):
 1. Add a Date Slicer.
 2. Add Card visual → New Measure → $\text{StartDate} = \text{MIN}(\text{DateTable}[\text{Date}])$
 3. Add Card visual → New Measure → $\text{EndDate} = \text{MAX}(\text{DateTable}[\text{Date}])$
 4. Connect slicer → it will update both cards.

■ Quick Quiz

Q: Which Slicer style shows a sliding bar to select a date range?

A: Style = 'Between' (Range slider).

■ 6.5 Input Boxes & Limitations in Power BI

■ The 2nd-Grader Explanation

Power BI is not Excel — you cannot just type numbers into a cell! Getting user input is tricky. Power BI uses What-If Parameters as 'input boxes' instead.

■ The 'Full Marks' Details

- Power BI does NOT have a native text/number input box like Excel.
- Workaround: Use What-If Parameter (Modeling → New Parameter).
 - Creates a slicer the user can slide to pick a number.
 - Creates a measure automatically representing the selected value.
- To ADD two input values (Task 5):
 1. Create Parameter1 (e.g., 0–100, increment 1).
 2. Create Parameter2 (same).
 3. New Measure: $\text{Sum} = [\text{Parameter1 Value}] + [\text{Parameter2 Value}]$
 4. Show Sum in a Card visual.
- LIMITATIONS of input in Power BI:
 1. No free-text input boxes.
 2. Only numeric values via What-If Parameters.
 3. No keyboard input directly into visuals.
 4. Input range must be predefined (min, max, step).

5. Not designed for data entry — use Power Apps for that.

■ Quick Quiz

Q: What Power BI feature simulates a numeric input box?

A: What-If Parameter (Modeling tab → New Parameter).

SECTION 7 — Advanced Tasks: Age Groups, Sales & Visual Quality

■ 7.1 Age Groups Using Nested IF vs SWITCH

■ The 2nd-Grader Explanation

Making age groups is like sorting kids at a birthday party: under 5 go to the toddler table, 5–12 go to the kids table, 13–17 teens table, 18+ adults. Both Nested IF and SWITCH can do this — SWITCH is cleaner.

■ The 'Full Marks' Details

- USING NESTED IF:

```
AgeGroup = IF([Age]<13, "Child",  
IF([Age]<18, "Teen",  
IF([Age]<30, "Young Adult",  
IF([Age]<60, "Adult", "Senior"))))
```

- USING SWITCH (TRUE mode) — RECOMMENDED:

```
AgeGroup = SWITCH(TRUE(),  
[Age] < 13, "Child",  
[Age] < 18, "Teen",  
[Age] < 30, "Young Adult",  
[Age] < 60, "Adult",  
"Senior")
```

- Both produce the SAME result — SWITCH is just easier to read.
- SWITCH(TRUE()) evaluates each condition top-to-bottom; first TRUE wins.

■ Quick Quiz

Q: Write the SWITCH(TRUE()) formula to categorise Age < 18 as 'Minor' and 18+ as 'Adult'.

A: SWITCH(TRUE(), [Age] < 18, "Minor", "Adult")

■ 7.2 Sales Amount Between Dates (CALCULATE + Variables)

■ The 2nd-Grader Explanation

This is the BIG exam task (Task 10). Think of it like a shopkeeper who says: 'Show me total sales only for orders received between my chosen dates.' CALCULATE is the magic function that adds a date filter condition.

■ The 'Full Marks' Details

STEP-BY-STEP:

1. Create DateTable from order_date using CALENDAR or CALENDARAUTO.
2. Add Year, Quarter, MonthYear columns to DateTable.
3. Create relationship: DateTable[Date] ↔ Sales[received_date]
4. Create StartDate variable (measure):

StartDate = MIN(DateTable[Date])
5. Create EndDate variable (measure):

EndDate = MAX(DateTable[Date])
6. Create Sales measure:

Sales = CALCULATE(

SUM(Sales[sales_amount]),

Sales[received_date] >= [StartDate],

Sales[received_date] <= [EndDate]

)
7. Total Sales (simple measure): TotalSales = SUM(Sales[sales_amount])
8. Add DateTable slicer → Sales measure auto-filters via CALCULATE.
9. Display Sales on a Bar or Line Chart visual.

KEY CONCEPT: CALCULATE(expression, filter1, filter2) — applies filters to the expression.

■ Quick Quiz

Q: What does CALCULATE do differently from SUM?

A: CALCULATE evaluates an expression (like SUM) but applies additional FILTER conditions you specify.

■ 7.3 Ugly, Bad & Wrong Visuals — What to Know

■ The 2nd-Grader Explanation

Imagine getting a crayon drawing vs a neat diagram — same info, but one is hard to read. In data viz, there are three types of bad visuals: Ugly (looks bad), Bad (confusing), and Wrong (mathematically incorrect).

■ The 'Full Marks' Details

- UGLY: Has aesthetic problems — but still shows correct data.
 - Example: Bright garish colours, mismatched fonts, cluttered background grid.
 - The data is correct, just unpleasant to look at.

- BAD: Has perception problems — confusing or misleading.
 - Example: Each bar has its own separate Y-axis scale → makes bars look equal when they are not.
 - User can be tricked into wrong conclusions.
- WRONG: Has mathematical errors — objectively incorrect.
 - Example: No Y-axis at all → bars appear to be lengths 1, 3, 2 when values are 3, 5, 4.
 - The numbers shown are literally wrong.
- Good visualisation principle: Accurately conveys data + aesthetically pleasing.
- 'Ugly' is more subjective; 'Bad' and 'Wrong' are more objective.

■ Quick Quiz

Q: What is the difference between a 'bad' and a 'wrong' figure?

A: BAD = misleading/confusing (perception issue). WRONG = mathematically incorrect (actual numbers are wrong).

■ FINAL CHEAT SHEET — Most Important Formulas

Formula / Function	What it does	Example
IF (cond, T, F)	Basic condition check	IF([Age]>=18,"Adult","Minor")
SWITCH (TRUE(), cond, val, ...)	Multi-condition (clean)	SWITCH(TRUE(), [Age]<13, "Child", "Adult")
DATEDIFF (S, E, DAY)	Days between two dates	DATEDIFF([Start],[End],DAY)
DATEDIFF (DOB, TODAY(), YEAR)	Age in years	DATEDIFF([DOB],TODAY(),YEAR)
FORMAT ([Date], "MMM YYYY")	Date as text format	FORMAT([Date],"MMM YYYY") → Mar 2025
CALENDAR (S, E)	Manual date table	CALENDAR(DATE(2020,1,1),TODAY())
CALENDARAUTO ()	Auto date table	CALENDARAUTO()
YEAR ([Date])	Extract year	YEAR([Date]) → 2025
MONTH ([Date])	Extract month number	MONTH([Date]) → 3
QUARTER ([Date])	Extract quarter	"Q" & QUARTER([Date]) → Q1
MIN ([Date])	Earliest/start date	StartDate = MIN(DateTable[Date])
MAX ([Date])	Latest/end date	EndDate = MAX(DateTable[Date])
CALCULATE (SUM(), filter)	SUM with condition	CALCULATE(SUM([Sales]),[Date]>=[Start])
SUM ([column])	Total sum	TotalSales = SUM([sales_amount])
AVERAGE ([column])	Average value	AvgTAT = AVERAGE([TAT_Days])

■ You have studied EVERY topic. Go into that exam with confidence.

You've got this! ■